

# Unveil Symfony AI

---

**Bringing AI Power to PHP Applications**



Feedback

# Hello!

## Mathieu Santostefano

Tech Expert **Sensio**Labs

 Symfony Core Team Member

 Clever Cloud Ambassador

 @welcomattic.com

 @welcomattic





SymfonyAI

# Initiative

---



# Initiative

---

 17 Announced July 2025



# Initiative

---

 17 Announced July 2025

 Mono-repository ``symfony/ai``



# Initiative

---

 17 Announced July 2025

 Mono-repository ``symfony/ai``

 **Symfony Standards**

Following Symfony best practices



# Initiative

---

 17 Announced July 2025

 Mono-repository ``symfony/ai``

 **Symfony Standards**

Following Symfony best practices

 **Experimental**

Active development, v0.5 tagged last week







# Main Goals

---



# Main Goals

---

- 🚀 Enable AI Features  
in PHP applications





# Main Goals

---



**Enable AI Features**

in PHP applications



**Not only LLMs**

Complete AI ecosystem integration



# Main Goals

---



## Enable AI Features

in PHP applications



## Not only LLMs

Complete AI ecosystem integration



## PHP Components Set

Modular and reusable





# Main Goals

---



## Enable AI Features

in PHP applications



## Not only LLMs

Complete AI ecosystem integration



## PHP Components Set

Modular and reusable



## Vendor Agnostic

Works with multiple AI providers

# Stats

---



# Stats

★ 80+ packages across 4 sub-projects





# Stats

★ 80+ **packages** across 4 sub-projects

👥 100+ **contributors** from the community





# Stats

- ★ 80+ **packages** across 4 sub-projects
- 👥 100+ **contributors** from the community
- 🇩🇪 2600+ **commits** since launch



# Stats

- ★ 80+ **packages** across 4 sub-projects
- 👥 100+ **contributors** from the community
- 🇩🇪 2600+ **commits** since launch
- 🌐 25+ **Platform Bridges** for AI providers



# Stats

- ★ 80+ **packages** across 4 sub-projects
- 👥 100+ **contributors** from the community
- 🇩🇪 2600+ **commits** since launch
- 🌐 25+ **Platform Bridges** for AI providers
- 🇮🇹 20+ **Store Bridges** for vector databases



# Stats

- ★ 80+ **packages** across 4 sub-projects
- 👥 100+ **contributors** from the community
- 🇩🇪 2600+ **commits** since launch
- 🌐 25+ **Platform Bridges** for AI providers
- 🇮🇹 20+ **Store Bridges** for vector databases
- 🔧 10+ **Tools** executable by AI agents





# Features

---



# Platform Component

---

# Platform Component

---

# Platform Component

---

## Abstraction layer

for AI Platforms: "where models run"



# Platform Component

---

## Abstraction layer

for AI Platforms: "where models run"

## Central Interface

for Model Inference

# Platform Component

---

## Abstraction layer

for AI Platforms: "where models run"

## Central Interface

for Model Inference

## Bridges

to connect 25+ AI Platforms

# Platform Component

---

## **Abstraction layer**

for AI Platforms: "where models run"

## **Central Interface**

for Model Inference

## **Bridges**

to connect 25+ AI Platforms

## **Multi-modal**

compatibility (text, image, audio, PDF)

# Platform Component

---

## **Abstraction layer**

for AI Platforms: "where models run"

## **Central Interface**

for Model Inference

## **Bridges**

to connect 25+ AI Platforms

## **Multi-modal**

compatibility (text, image, audio, PDF)

## **Event system**

to hook into the lifecycle

# Platform Component

---

## Abstraction layer

for AI Platforms: "where models run"

## Central Interface

for Model Inference

## Bridges

to connect 25+ AI Platforms

## Multi-modal

compatibility (text, image, audio, PDF)

## Event system

to hook into the lifecycle

```
composer require symfony/ai-platform
```

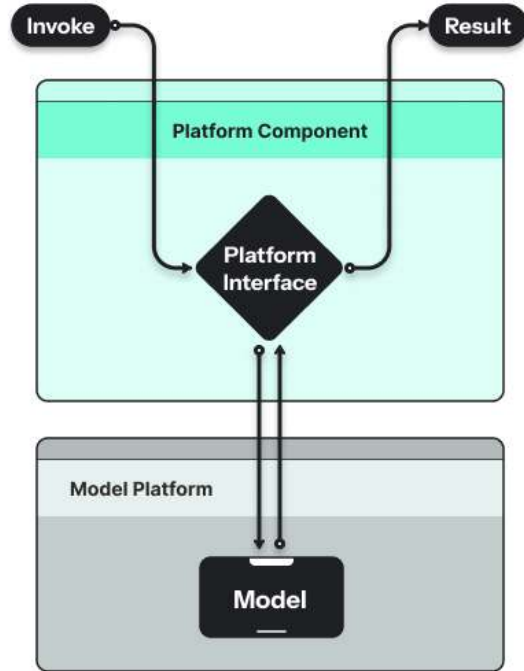


# Platform Component Architecture

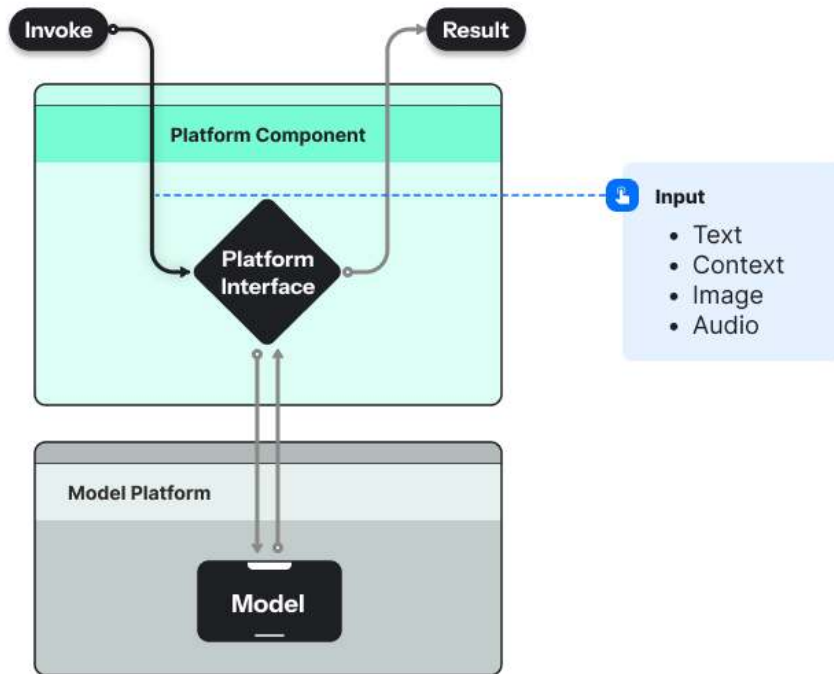
---

# Platform Component Architecture

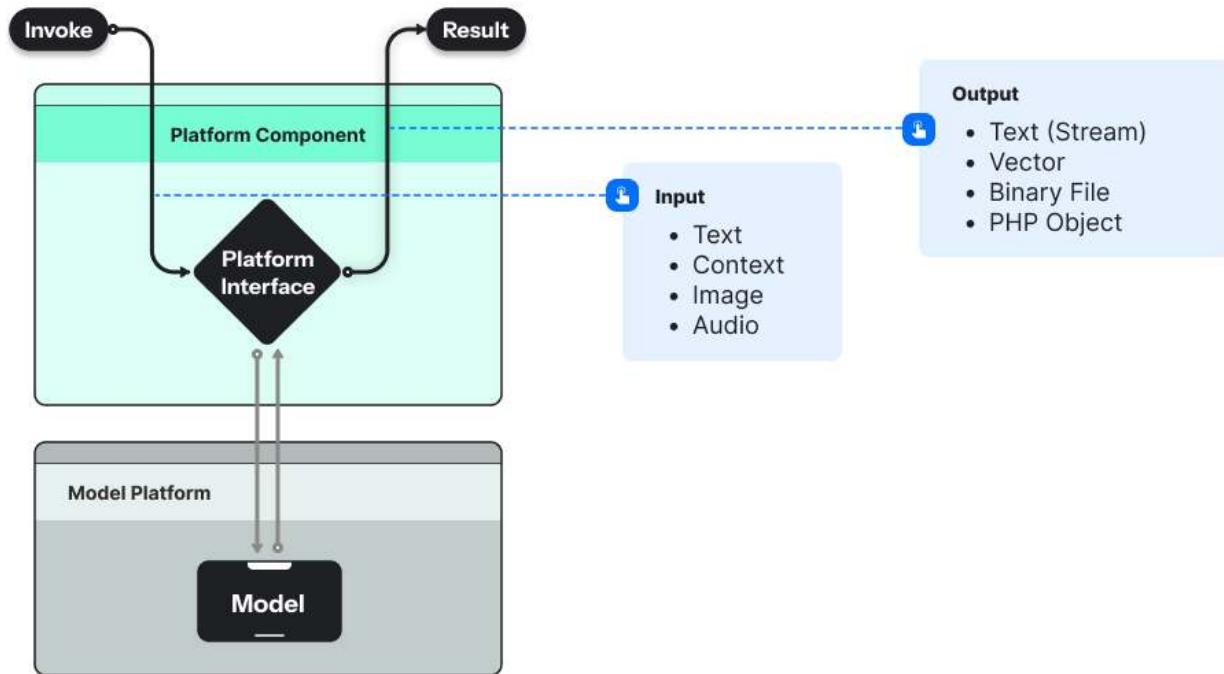
---



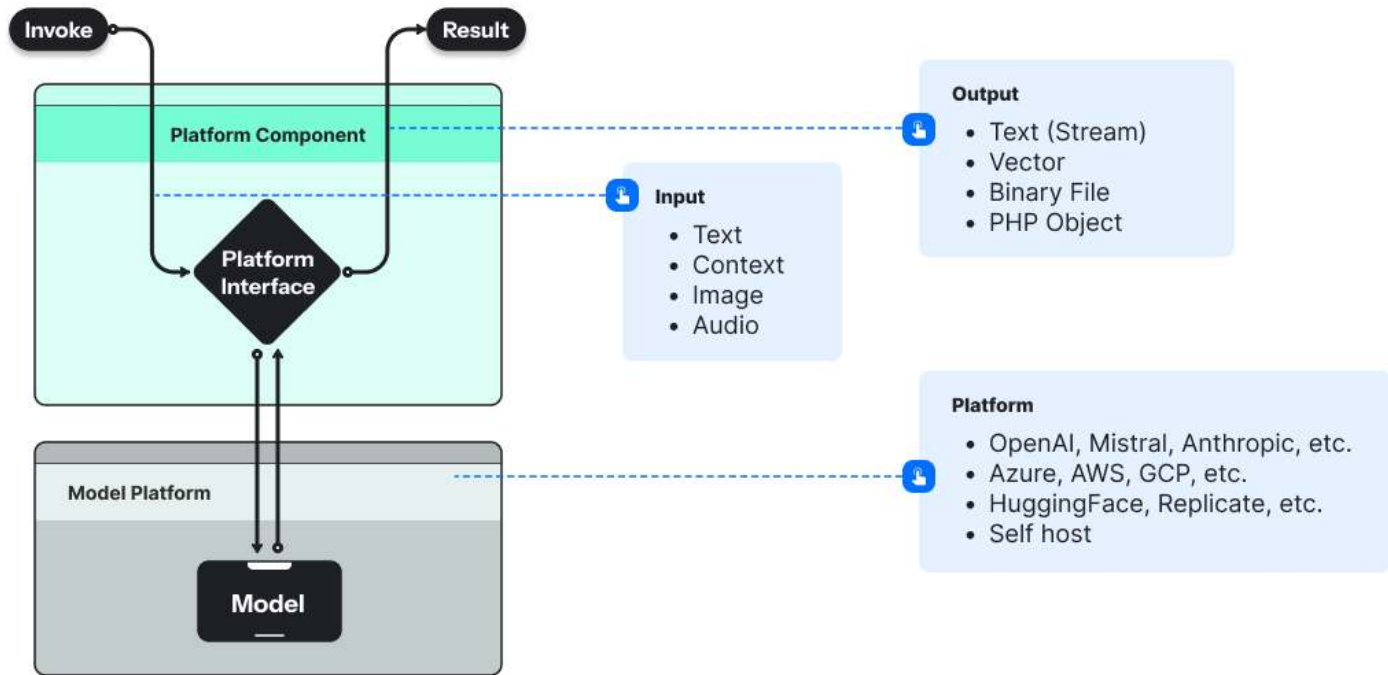
# Platform Component Architecture



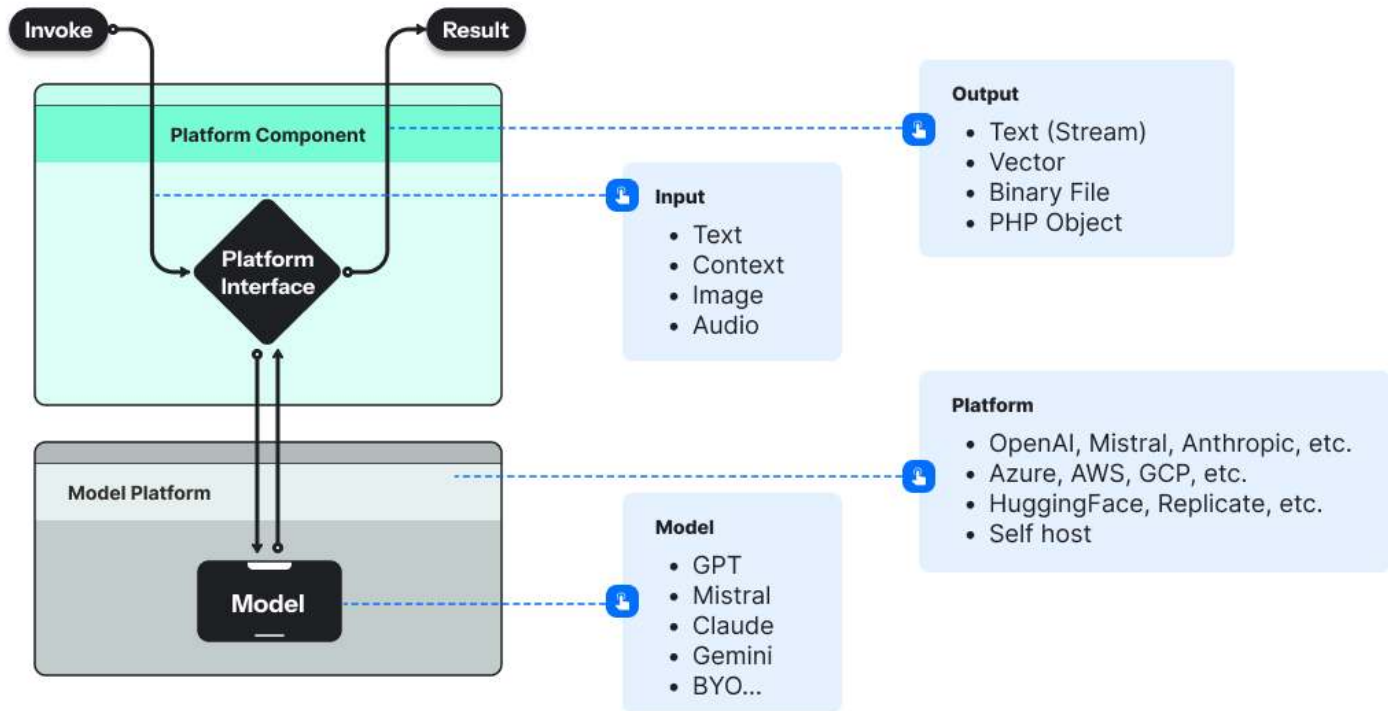
# Platform Component Architecture



# Platform Component Architecture



# Platform Component Architecture





# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# Model Inference

---

*(aka "run a prompt on a model")*

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```

# OpenAI Platform / GPT 5 mini Model

---

```
use Symfony\AI\Platform\Bridge\OpenAi\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gpt-5-mini', $input);

echo $result->asText();
```



# Anthropic Platform / Claude Sonnet 4.5 Model

---

```
use Symfony\AI\Platform\Bridge\Anthropic\PlatformFactory;
use Symfony\AI\Platform\Message\{Message, MessageBag};

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('claude-sonnet-4-5-20250929', $input);

echo $result->asText();
```

# Google Gemini Platform / Gemini 3 Pro Model

---

```
use Symfony\AI\Platform\Bridge\Gemini\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('gemini-3-pro', $input);

echo $result->asText();
```

# Mistral Platform / Mistral Large Model

---

```
use Symfony\AI\Platform\Bridge\Mistral\PlatformFactory;
use Symfony\AI\Platform\Message\Message, MessageBag;

$platform = PlatformFactory::create($apiKey);

$input = new MessageBag(
    Message::forSystem('You are a pirate and you write funny.'),
    Message::ofUser('What is the Symfony framework?'),
);

$result = $platform->invoke('mistral-large-latest', $input);

echo $result->asText();
```

# Streaming

*(to handle response token per token)*

```
$input = new MessageBag(  
    Message::forSystem('You are a pirate and you write funny.'),  
    Message::ofUser('What is the Symfony framework?'),  
);  
  
$result = $platform->invoke('mistral-large-latest', $input, [  
    'stream' => true,  
]);  
  
foreach ($result->asStream() as $word) {  
    echo $word;  
}
```

# Audio Input

---

## Multi-modal: Process audio files

```
$input = new MessageBag(  
  Message::ofUser(  
    'Describe the audio file.',  
    Audio::fromDataUrl('data:audio/mpeg;base64,/9j/4AAQ...'),  
  ),  
);  
  
$result = $platform->invoke('gemini-2.5-flash', $input);  
  
echo $result->asText();
```

# Image Input

---

## Multi-modal: Analyze images

```
$input = new MessageBag(  
    Message::ofUser(  
        'What is visible on the image?',  
        new ImageUrl('https://example.org/elephant.jpg'),  
    ),  
);  
  
$result = $platform->invoke('gemini-2.5-flash', $input);  
  
echo $result->asText();
```

# PDF Input

---

## Multi-modal: Extract from documents

```
$input = new MessageBag(  
    Message::ofUser(  
        'Describe the content of the PDF file.',  
        Document::fromFile('/path/to/document.pdf'),  
    ),  
);  
  
$result = $platform->invoke('gemini-2.5-flash', $input);  
  
echo $result->asText();
```

# Structured Output

---

*(to manipulate output programmatically)*

```
$input = new MessageBag(  
    Message::forSystem('Help users as math tutor, step by step.'),  
    Message::ofUser('how can I solve  $8x + 7 = -23$ '),  
);  
  
$result = $platform->invoke('mistral-small-latest', $messages, [  
    'response_format' => MathReasoning::class,  
]);  
  
echo $result->asObject();
```



# Structured Output Example

```
→ php mistral/structured-output-math.php
^ Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\MathReasoning^ {#322
  +steps: array:5 [
    0 => Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\Step^ {#341
      +explanation: "First, we need to isolate the term with the variable x. To do this, we subtract 7 from both sides of the equation."
      +output: "8x + 7 - 7 = -23 - 7"
    }
    1 => Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\Step^ {#344
      +explanation: "Simplifying both sides gives us:"
      +output: "8x = -30"
    }
    2 => Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\Step^ {#343
      +explanation: "Next, we divide both sides by 8 to solve for x."
      +output: "8x / 8 = -30 / 8"
    }
    3 => Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\Step^ {#342
      +explanation: "Simplifying both sides gives us the value of x:"
      +output: "x = -30 / 8"
    }
    4 => Symfony\AI\Platform\Tests\Fixtures\StructuredOutput\Step^ {#350
      +explanation: "Simplifying the fraction gives us:"
      +output: "x = -15 / 4"
    }
  ]
  +finalAnswer: "The solution to the equation 8x + 7 = -23 is x = -15/4."
  +result: -3.75
}
```

# Embedded Models

---

*(to transform content into mathematical vectors)*

```
$result = $platform->invoke('text-embedding-3-small', $text);  
  
echo $result->asVector(); // array of floats
```

# Embedded Models

---

*(to transform content into mathematical vectors)*

```
$result = $platform->invoke('text-embedding-3-small', $text);  
  
echo $result->asVector(); // array of floats
```

✨ Perfect for **semantic search** and **RAG** (Retrieval-Augmented Generation) applications

# Hugging Face Integration 🤗

*(trigger different tasks like NLP, Computer Vision, Robotics, ...)*

```
use Symfony\AI\Platform\Bridge\HuggingFace\PlatformFactory;
use Symfony\AI\Platform\Bridge\HuggingFace\Task;

$platform = PlatformFactory::create($apiKey);
$result = $platform->invoke('facebook/detr-resnet-50', $image, [
    'task' => Task::OBJECT_DETECTION,
]);

echo $result->asObject(); // image classification result
```

# Hugging Face Integration 🤗

*(trigger different tasks like NLP, Computer Vision, Robotics, ...)*

```
use Symfony\AI\Platform\Bridge\HuggingFace\PlatformFactory;
use Symfony\AI\Platform\Bridge\HuggingFace\Task;

$platform = PlatformFactory::create($apiKey);
$result = $platform->invoke('facebook/detr-resnet-50', $image, [
    'task' => Task::OBJECT_DETECTION,
]);

echo $result->asObject(); // image classification result
```

# Hugging Face Integration 🤗

*(trigger different tasks like NLP, Computer Vision, Robotics, ...)*

```
use Symfony\AI\Platform\Bridge\HuggingFace\PlatformFactory;
use Symfony\AI\Platform\Bridge\HuggingFace\Task;

$platform = PlatformFactory::create($apiKey);
$result = $platform->invoke('facebook/detr-resnet-50', $image, [
    'task' => Task::OBJECT_DETECTION,
]);

echo $result->asObject(); // image classification result
```

# Hugging Face Integration 🤗

*(trigger different tasks like NLP, Computer Vision, Robotics, ...)*

```
use Symfony\AI\Platform\Bridge\HuggingFace\PlatformFactory;
use Symfony\AI\Platform\Bridge\HuggingFace\Task;

$platform = PlatformFactory::create($apiKey);
$result = $platform->invoke('facebook/detr-resnet-50', $image, [
    'task' => Task::OBJECT_DETECTION,
]);

echo $result->asObject(); // image classification result
```

# Hugging Face Integration 🤗

*(trigger different tasks like NLP, Computer Vision, Robotics, ...)*

```
use Symfony\AI\Platform\Bridge\HuggingFace\PlatformFactory;
use Symfony\AI\Platform\Bridge\HuggingFace\Task;

$platform = PlatformFactory::create($apiKey);
$result = $platform->invoke('facebook/detr-resnet-50', $image, [
    'task' => Task::OBJECT_DETECTION,
]);

echo $result->asObject(); // image classification result
```

🚀 Integration with Hugging Face's **Inference Hub API** - Access thousands of open-source models



# 25+ Platform Bridges available

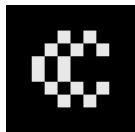
---

## Native Platforms

Run provider's models on provider's infrastructure



Anthropic



Cartesia



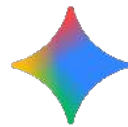
Decart



DeepSeek



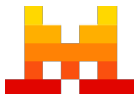
ElevenLabs



Gemini



Meta



Mistral



OpenAI



Perplexity



Voyage

# 25+ Platform Bridges available

---

## Model Runner Platforms (Model-as-a-Service)

Run models on other cloud infrastructure



AI/ML API



Albert



Azure AI



Bedrock



Cerebras



HuggingFace



Replicate



OpenRouter



Scaleway AI



VertexAI

# 25+ Platform Bridges available

---

## Self-Hosted Platforms

Run models on your own infrastructure



Ollama



Docker Model Runner



LiteLLM



LM Studio



# **Agent Component**

---

# Agent Component

---

# Agent Component

---



**Multi-step Interaction** with AI Models

# Agent Component

---

 **Multi-step Interaction** with AI Models

 **Combination of Components:**

Platform & Models

Messages / Prompts as Context

Memory for conversations

Tools for external actions

Event-driven workflows

# Agent Component

---

 **Multi-step Interaction** with AI Models

 **Combination of Components:**

Platform & Models

Messages / Prompts as Context

Memory for conversations

Tools for external actions

Event-driven workflows

 **Various Architectures**

From static workflows to autonomous orchestration



# Agent Component

---

 **Multi-step Interaction** with AI Models

 **Combination of Components:**

Platform & Models

Messages / Prompts as Context

Memory for conversations

Tools for external actions

Event-driven workflows

```
composer require symfony/ai-agent
```

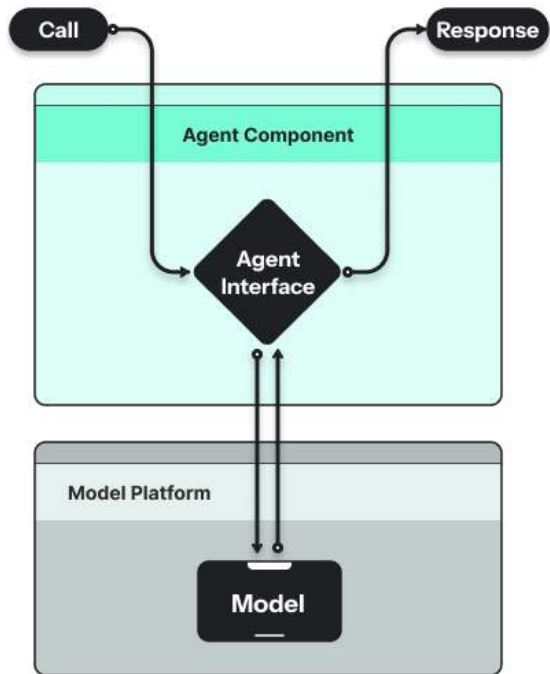
 **Various Architectures**

From static workflows to autonomous orchestration

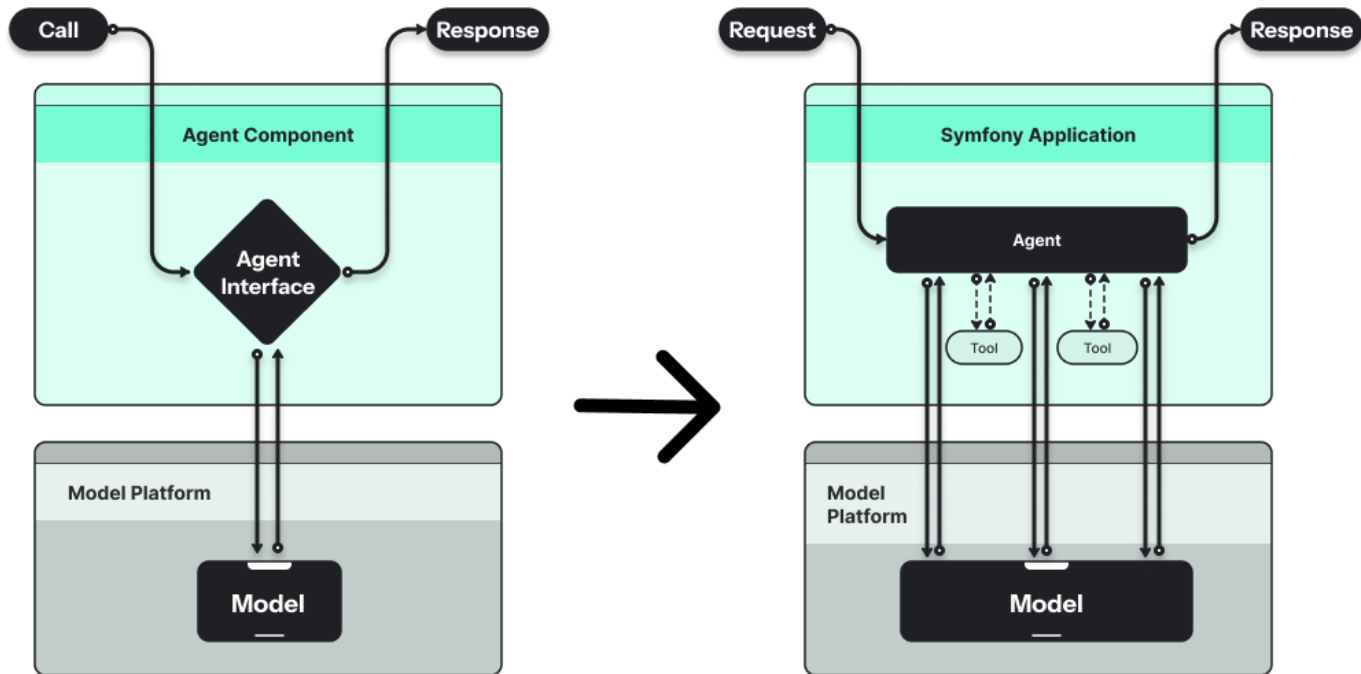
# Agent Component Architecture

---

# Agent Component Architecture

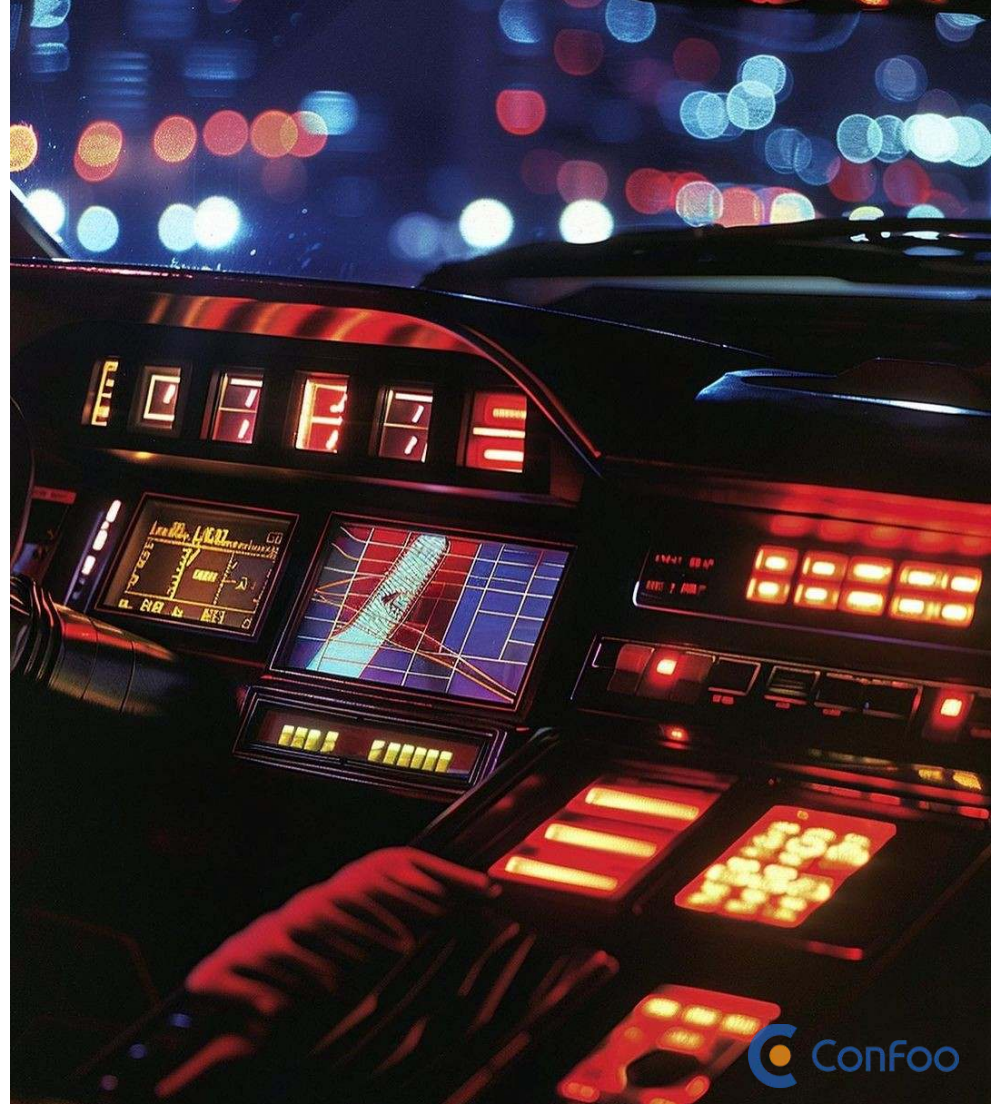


# Agent Component Architecture



# Tool Calling Flow

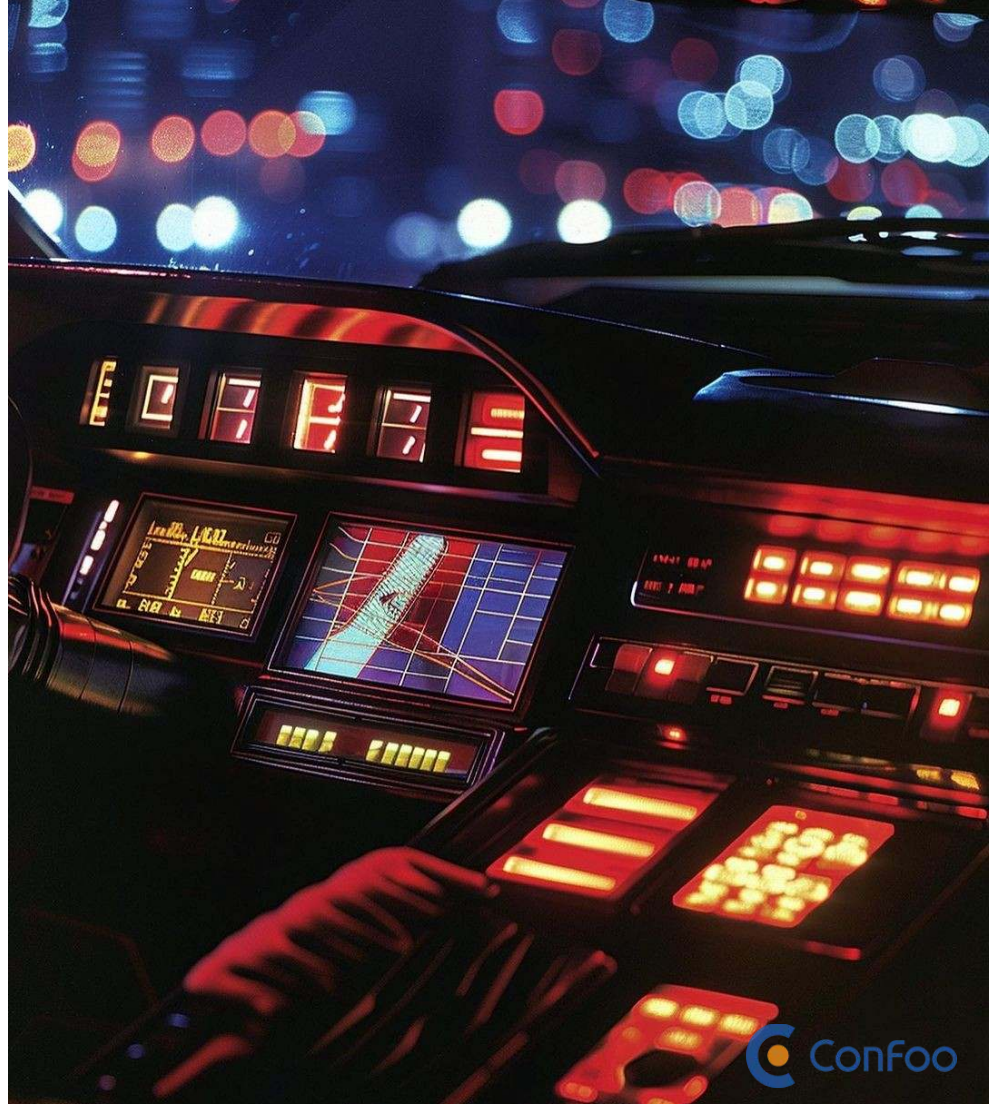
---



# Tool Calling Flow

---

1. **Agent provides Tools** to the Model  
Exposed to Model as JSON Schema definition

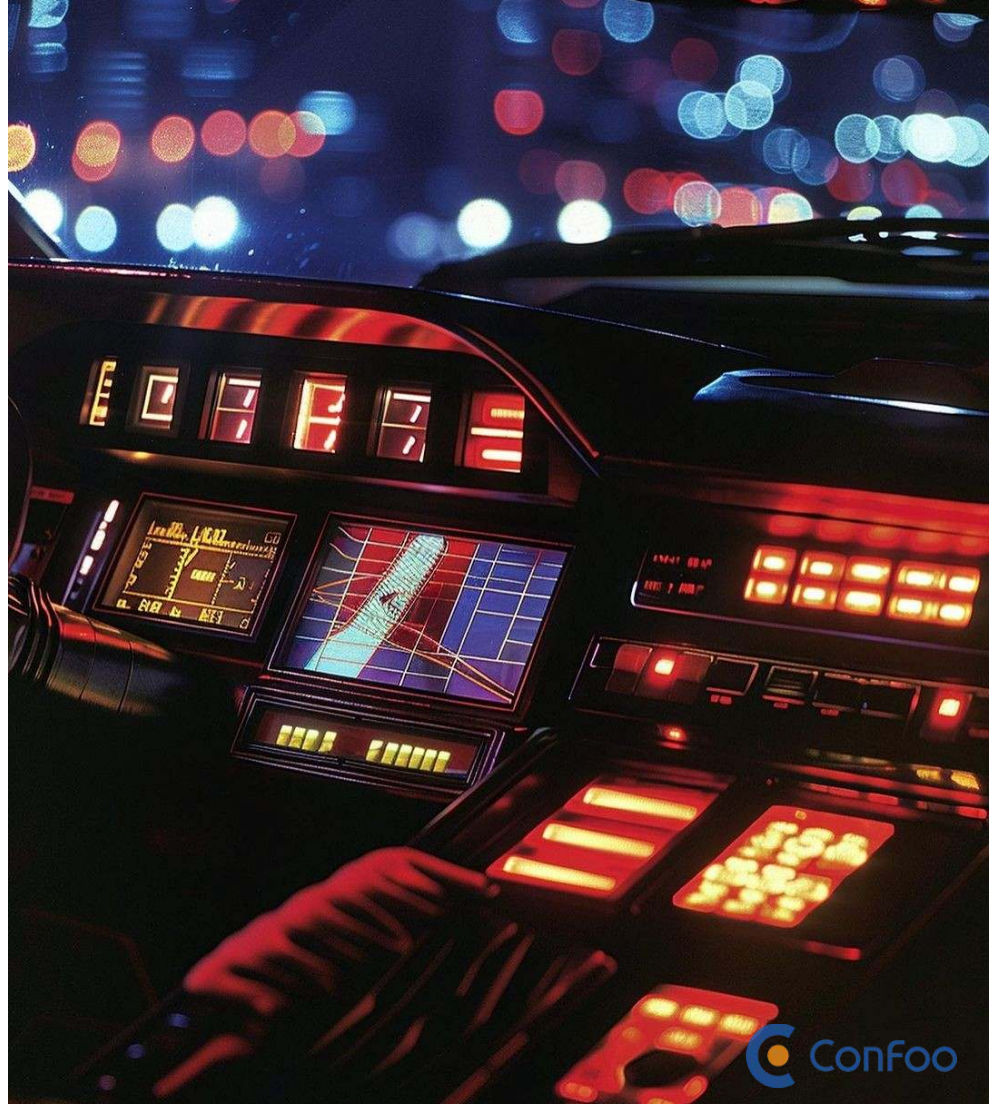




# Tool Calling Flow

---

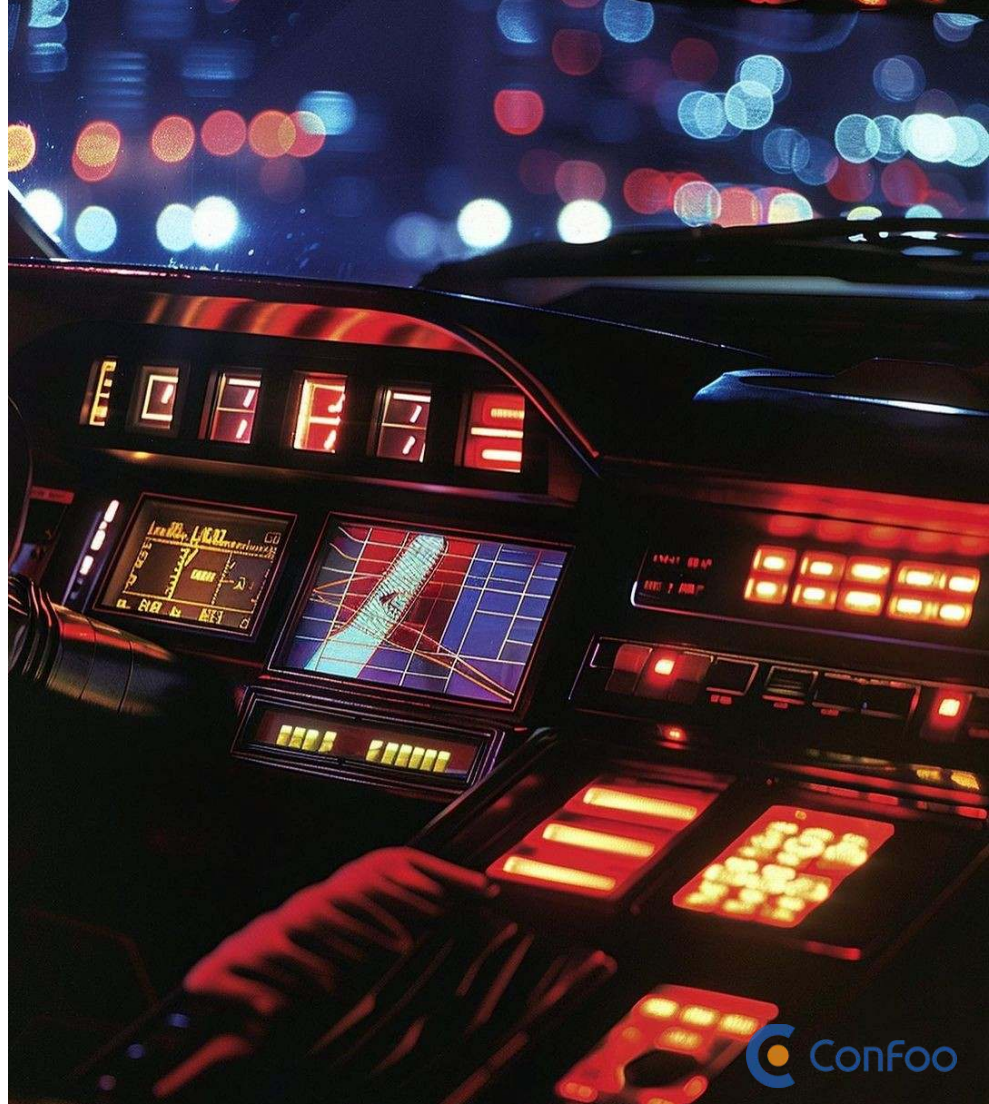
1. **Agent provides Tools** to the Model  
Exposed to Model as JSON Schema definition
2. **Model invokes Tools** by name with arguments  
Intermediate step in the conversation flow



# Tool Calling Flow

---

1. **Agent provides Tools** to the Model  
Exposed to Model as JSON Schema definition
2. **Model invokes Tools** by name with arguments  
Intermediate step in the conversation flow
3. **Tool execution** by the Agent  
Run PHP Code, run a CLI, call an API, etc.

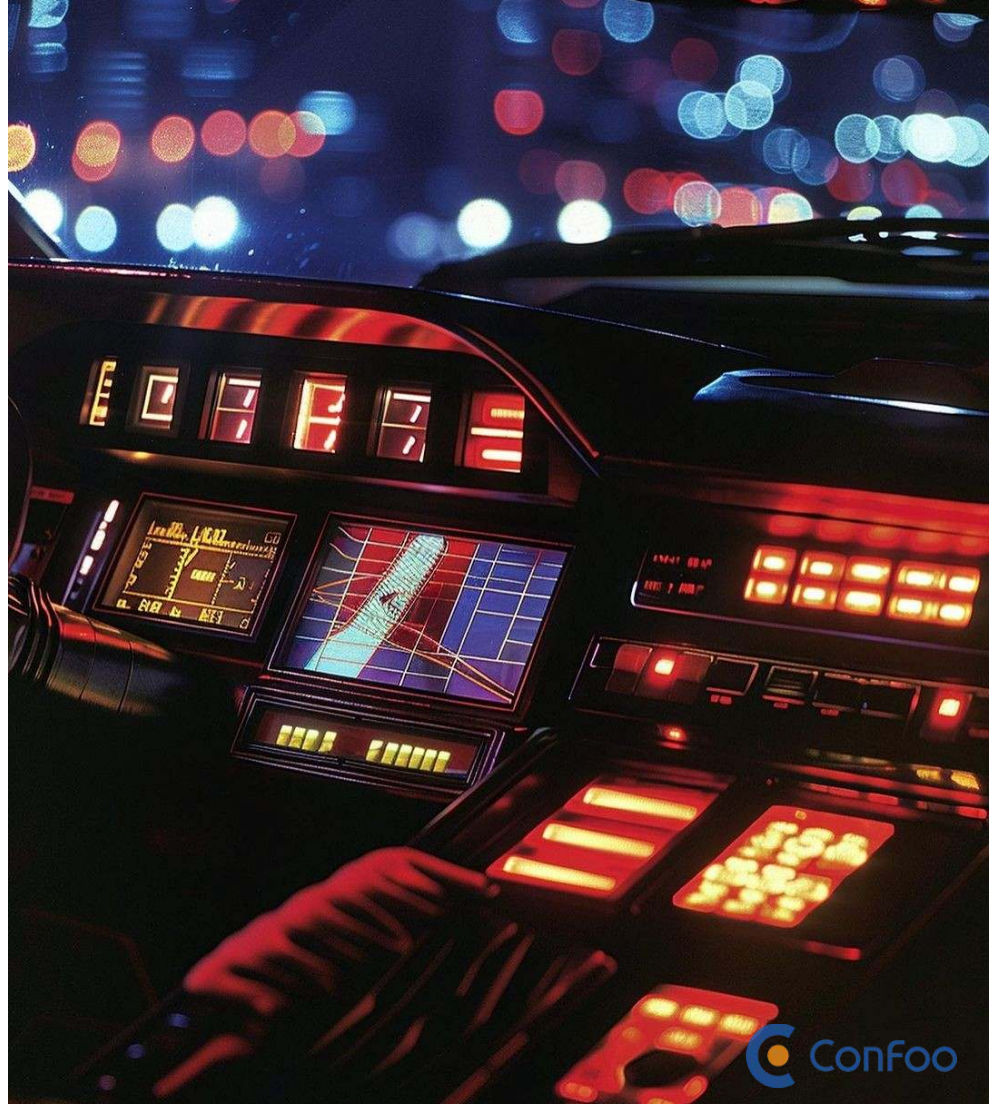




# Tool Calling Flow

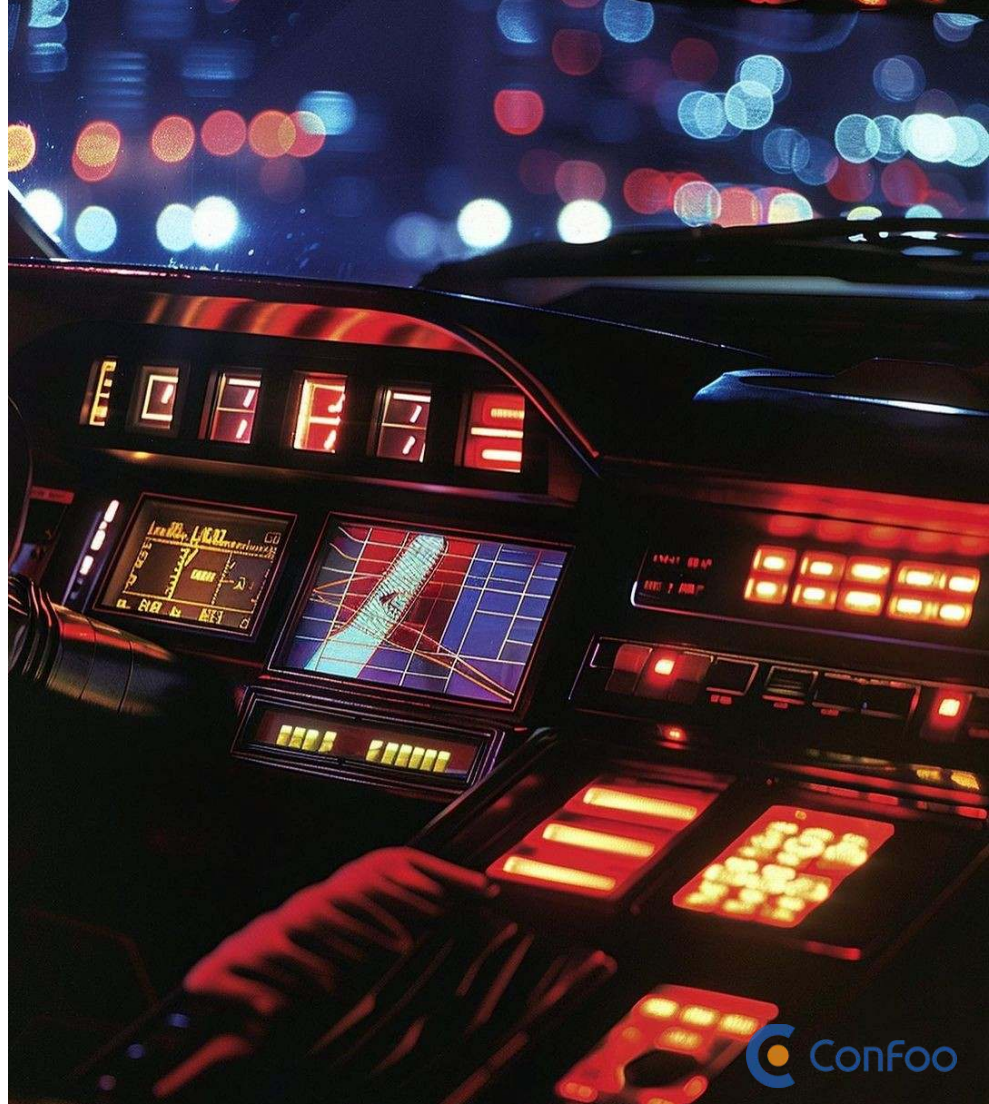
---

1. **Agent provides Tools** to the Model  
Exposed to Model as JSON Schema definition
2. **Model invokes Tools** by name with arguments  
Intermediate step in the conversation flow
3. **Tool execution** by the Agent  
Run PHP Code, run a CLI, call an API, etc.
4. **Result sent back** to the Model  
Model uses result to continue conversation



# Tool Calling Flow

1. **Agent provides Tools** to the Model  
Exposed to Model as JSON Schema definition
2. **Model invokes Tools** by name with arguments  
Intermediate step in the conversation flow
3. **Tool execution** by the Agent  
Run PHP Code, run a CLI, call an API, etc.
4. **Result sent back** to the Model  
Model uses result to continue conversation
5. **Instrumentation** throughout the process  
Log and analyze all Tool calls and results



# How Does It Work? 🎯

---

```
bin/console ai:agent:call default
```

# Tool Calling Example

```
use Symfony\AI\Agent\Toolbox\Attribute\AsTool;
use Symfony\Component\Clock\ClockInterface;

#[AsTool('cookbook_create_recipe', 'Creates a new recipe in the cookbook.')]
final readonly class CookbookCreateRecipe
{
    public function __construct(
        private EntityManagerInterface $em,
    ) {}

    public function __invoke(Recipe $recipe): string
    {
        // validate & persist recipe ...
        return 'Recipe created successfully!';
    }
}
```

# Tool Calling Example

---

```
use Symfony\AI\Agent\Toolbox\Attribute\AsTool;
use Symfony\Component\Clock\ClockInterface;

#[AsTool('cookbook_create_recipe', 'Creates a new recipe in the cookbook.')]
final readonly class CookbookCreateRecipe
{
    public function __construct(
        private EntityManagerInterface $em,
    ) {}

    public function __invoke(Recipe $recipe): string
    {
        // validate & persist recipe ...
        return 'Recipe created successfully!';
    }
}
```

# Tool Calling Example

---

```
use Symfony\AI\Agent\Toolbox\Attribute\AsTool;
use Symfony\Component\Clock\ClockInterface;

#[AsTool('cookbook_create_recipe', 'Creates a new recipe in the cookbook.')]
final readonly class CookbookCreateRecipe
{
    public function __construct(
        private EntityManagerInterface $em,
    ) {}

    public function __invoke(Recipe $recipe): string
    {
        // validate & persist recipe ...
        return 'Recipe created successfully!';
    }
}
```

# Tool Calling Example

---

```
use Symfony\AI\Agent\Toolbox\Attribute\AsTool;  
use Symfony\Component\Clock\ClockInterface;  
  
#[AsTool('cookbook_create_recipe', 'Creates a new recipe in the cookbook.')]  
final readonly class CookbookCreateRecipe  
{  
    public function __construct(  
        private EntityManagerInterface $em,  
    ) {}  
  
    public function __invoke(Recipe $recipe): string  
    {  
        // validate & persist recipe ...  
        return 'Recipe created successfully!';  
    }  
}
```

# Tool Calling Example

```
use Symfony\AI\Agent\Toolbox\Attribute\AsTool;
use Symfony\Component\Clock\ClockInterface;

#[AsTool('cookbook_create_recipe', 'Creates a new recipe in the cookbook.')]
final readonly class CookbookCreateRecipe
{
    public function __construct(
        private EntityManagerInterface $em,
    ) {}

    public function __invoke(Recipe $recipe): string
    {
        // validate & persist recipe ...
        return 'Recipe created successfully!';
    }
}
```



# Agent Tool Bridges

---

**10+ Tool Bridges available**



Brave



Clock



Firecrawl



Mapbox



OpenMeteo



Scraper



Serpapi



Tavily

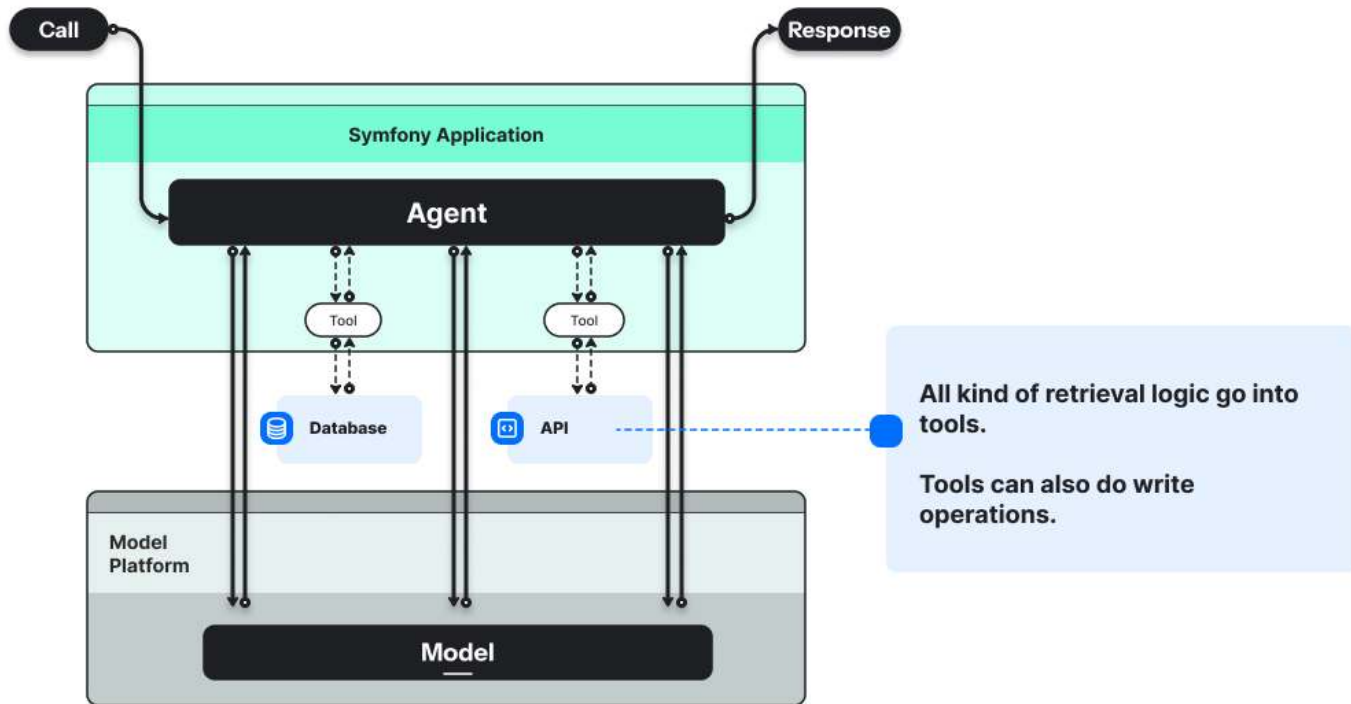


Wikipedia



YouTube

# Agentic Application Architecture





# **MCP - Model Context Protocol**

---



# MCP

🔌 Open Standard

by Anthropic (Nov 2024)





# MCP

## 🔌 Open Standard

by Anthropic (Nov 2024)

## 🌐 Universal Adapter for AI

like USB-C for devices



# MCP

## 🔌 Open Standard

by Anthropic (Nov 2024)

## 🌐 Universal Adapter for AI

like USB-C for devices

## 🤝 Standardized Integration

between LLMs and external tools





# MCP

---

## 🔌 Open Standard

by Anthropic (Nov 2024)

## 🌐 Universal Adapter for AI

like USB-C for devices

## 🤝 Standardized Integration

between LLMs and external tools

## 🏢 Industry Adoption

OpenAI, Google, Anthropic, Mistral





# MCP

## 🔌 Open Standard

by Anthropic (Nov 2024)

## 🌐 Universal Adapter for AI

like USB-C for devices

## 🤝 Standardized Integration

between LLMs and external tools

## 🏢 Industry Adoption

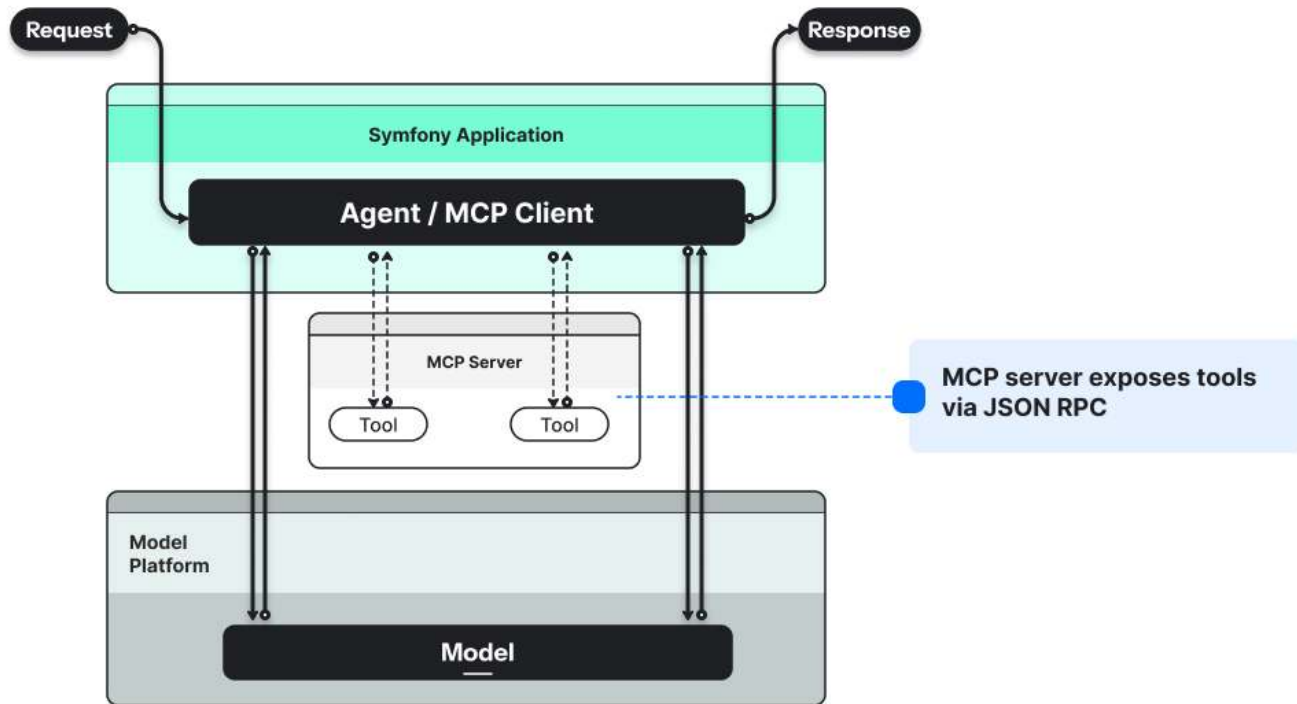
OpenAI, Google, Anthropic, Mistral

## 🔧 Expose Capabilities

Tools, Resources, Prompts



# MCP Client / Server integration



# MCP Architecture

---

# MCP Architecture

---

## MCP Server Capabilities

| Capability       | Description                                            |
|------------------|--------------------------------------------------------|
| <b>Tools</b>     | Functions the model can invoke (API calls, DB queries) |
| <b>Resources</b> | Data sources the model can read (files, configs)       |
| <b>Prompts</b>   | Pre-defined prompt templates for common tasks          |

# Official PHP SDK

---

# Official PHP SDK

---

## Framework-agnostic

Works with any PHP project

# Official PHP SDK

---

## Framework-agnostic

Works with any PHP project

## PHP 8 Attributes

```
`#[McpTool]`, `#[McpResource]`, `#[McpPrompt]`
```

# Official PHP SDK

---

## Framework-agnostic

Works with any PHP project

## PHP 8 Attributes

```
`#[McpTool]` , `#[McpResource]` , `#[McpPrompt]`
```

## Auto-discovery

Scans directories for MCP elements



# Official PHP SDK

---

## Framework-agnostic

Works with any PHP project

## PHP 8 Attributes

```
`#[McpTool]` , `#[McpResource]` , `#[McpPrompt]`
```

## Auto-discovery

Scans directories for MCP elements

## Transport options

STDIO and Streamable HTTP support

# Official PHP SDK

---

## Framework-agnostic

Works with any PHP project

## PHP 8 Attributes

```
`#[McpTool]`, `#[McpResource]`, `#[McpPrompt]`
```

## Auto-discovery

Scans directories for MCP elements

## Transport options

STDIO and Streamable HTTP support

```
composer require mcp/sdk
```

GitHub: [github.com/modelcontextprotocol/php-sdk](https://github.com/modelcontextprotocol/php-sdk)

# Creating an MCP Server (STDIO Transport)

---

```
#!/usr/bin/env php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use Mcp\Server;
use Mcp\Server\Transport\StdioTransport;

$server = Server::builder()
    ->setServerInfo('Calculator Server', '1.0.0')
    ->setDiscovery(__DIR__, ['.'])
    ->build();

$transport = new StdioTransport();

$server->run($transport);
```

# Creating an MCP Server (STDIO Transport)

---

```
#!/usr/bin/env php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use Mcp\Server;
use Mcp\Server\Transport\StdioTransport;

$server = Server::builder()
    ->setServerInfo('Calculator Server', '1.0.0')
    ->setDiscovery(__DIR__, ['.'])
    ->build();

$transport = new StdioTransport();

$server->run($transport);
```

# Creating an MCP Server (STDIO Transport)

---

```
#!/usr/bin/env php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use Mcp\Server;
use Mcp\Server\Transport\StdioTransport;

$server = Server::builder()
    ->setServerInfo('Calculator Server', '1.0.0')
    ->setDiscovery(__DIR__, ['.'])
    ->build();

$transport = new StdioTransport();

$server->run($transport);
```

# Creating an MCP Server (STDIO Transport)

---

```
#!/usr/bin/env php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use Mcp\Server;
use Mcp\Server\Transport\StdioTransport;

$server = Server::builder()
    ->setServerInfo('Calculator Server', '1.0.0')
    ->setDiscovery(__DIR__, ['.'])
    ->build();

$transport = new StdioTransport();

$server->run($transport);
```

# Creating an MCP Server (STDIO Transport)

---

```
#!/usr/bin/env php
<?php

require_once __DIR__ . '/vendor/autoload.php';

use Mcp\Server;
use Mcp\Server\Transport\StdioTransport;

$server = Server::builder()
    ->setServerInfo('Calculator Server', '1.0.0')
    ->setDiscovery(__DIR__, ['.'])
    ->build();

$transport = new StdioTransport();

$server->run($transport);
```

# Creating Tools (Code that model can invoke)

---

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```



# Creating Tools (Code that model can invoke)

---

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```

# Creating Tools (Code that model can invoke)

---

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```

# Creating Tools (Code that model can invoke)

---

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```

# Creating Tools (Code that model can invoke)

---

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```

# Creating Tools (Code that model can invoke)

```
use Mcp\Capability\Attribute\McpTool;

class WeatherTools
{
    #[McpTool(name: 'get_weather', description: 'Get current weather for a city')]
    public function getWeather(string $city): array
    {
        // Call weather API...
        return ['condition' => $condition, 'temperature' => $temperature];
    }
}
```

✅ The model can now call `get_weather` with a city name and receive structured data back

# Creating Resources (Data the model can read)

---

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```

# Creating Resources (Data the model can read)

---

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```

# Creating Resources (Data the model can read)

---

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```



# Creating Resources (Data the model can read)

---

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```

# Creating Resources (Data the model can read)

---

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```

# Creating Resources (Data the model can read)

```
use Mcp\Capability\Attribute\McpResource;

class ConfigResources
{
    #[McpResource(uri: 'config://app/settings', name: 'App Settings')]
    public function getSettings(): array
    {
        return [
            'app_name' => 'My Application',
            'version' => '2.0.0',
        ];
    }
}
```

📖 Resources use **URI schemes** - the model reads `config://app/settings` to get app configuration

# Resource Templates (Dynamic resources)

---

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```

# Resource Templates (Dynamic resources)

---

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```

# Resource Templates (Dynamic resources)

---

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```

# Resource Templates (Dynamic resources)

---

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```

# Resource Templates (Dynamic resources)

---

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```



# Resource Templates (Dynamic resources)

```
use Mcp\Capability\Attribute\McpResourceTemplate;

class UserResources
{
    #[McpResourceTemplate(uriTemplate: 'user://{id}/profile', name: 'User Profile')]
    public function getUserProfile(int $id): array
    {
        // Fetch user from database...
        return ['name' => $user->getName(), 'email' => $user->getEmail()];
    }
}
```

🔗 Uses RFC 6570 URI Template syntax - model requests ``user://{42}/profile`` to get user #42

# Creating Prompts templates

---

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

# Creating Prompts templates

---

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

# Creating Prompts templates

---

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

# Creating Prompts templates

---

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

# Creating Prompts templates

---

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

# Creating Prompts templates

```
use Mcp\Capability\Attribute\McpPrompt;

class PromptTemplates
{
    #[McpPrompt(name: 'code_review', description: 'Review code for best practices')]
    public function codeReviewPrompt(string $language, string $code): array
    {
        return ['messages' => [
            ['role' => 'user', 'content' => "Review this {$language} code:\n\n{$code}"]
        ]];
    }
}
```

 Prompts provide **reusable templates** - clients can request `code_review` with parameters

# Complete Server Example

---

```
// autoloader require, use statements...
class Calculator
{
    #[McpTool(name: 'add', description: 'Add two numbers')]
    public function add(int $a, int $b): int { return $a + $b; }

    #[McpTool(name: 'multiply', description: 'Multiply two numbers')]
    public function multiply(int $a, int $b): int { return $a * $b; }
}

Server::make()
    ->withServerInfo('Calculator', '1.0.0')
    ->withDiscovery(__DIR__, ['.'])
    ->build()
    ->connect(new StdioTransport());
```



# Complete Server Example

---

```
// autoloader require, use statements...
class Calculator
{
    #[McpTool(name: 'add', description: 'Add two numbers')]
    public function add(int $a, int $b): int { return $a + $b; }

    #[McpTool(name: 'multiply', description: 'Multiply two numbers')]
    public function multiply(int $a, int $b): int { return $a * $b; }
}

Server::make()
    ->withServerInfo('Calculator', '1.0.0')
    ->withDiscovery(__DIR__, ['.'])
    ->build()
    ->connect(new StdioTransport());
```

# Complete Server Example

---

```
// autoloader require, use statements...
class Calculator
{
    #[McpTool(name: 'add', description: 'Add two numbers')]
    public function add(int $a, int $b): int { return $a + $b; }

    #[McpTool(name: 'multiply', description: 'Multiply two numbers')]
    public function multiply(int $a, int $b): int { return $a * $b; }
}

Server::make()
    ->withServerInfo('Calculator', '1.0.0')
    ->withDiscovery(__DIR__, ['.'])
    ->build()
    ->connect(new StdioTransport());
```

# **MCP Ecosystem**

---

## **Official GitHub Organization**

[github.com/modelcontextprotocol](https://github.com/modelcontextprotocol)

## **Specification & Docs**

[modelcontextprotocol.io](https://modelcontextprotocol.io)

# Chat Component

---

# Chat Component

---

# Chat Component

---

 **Build Chats** with Agents

 **Built on top of Agent component:**

 **With memory using dedicated Store Bridges**

# Chat Component

---

💬 **Build Chats** with Agents

🏗️ **Built on top of Agent component:**

🧠 **With memory using dedicated Store Bridges**



Cache



Cloudflare KV



Doctrine DBAL



Meilisearch



MongoDB



Redis



Session



SurrealDB

**POGOCACHE**

Pogocache

# Chat Component

💬 Build Chats with Agents

🏗️ Built on top of Agent component:

🧠 With memory using dedicated Store Bridges



Cache



Cloudflare KV



Doctrine DBAL



Meilisearch

```
composer require symfony/ai-chat
```



MongoDB



Redis



Session



SurrealDB

**POGOCACHE**

Pogocache



# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer questions.'));

$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer'));

$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer questions.'));

$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer questions.'));

$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer questions.'));

$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

# Basic Usage

```
use Symfony\AI\Agent\Agent;
use Symfony\AI\Chat\{Chat, InMemory\Store as InMemoryStore};
use Symfony\AI\Platform\{Bridge\Mistral\PlatformFactory, Message\Message};

$platform = PlatformFactory::create($apiKey);
$connection = DriverManager::getConnection(['driver' => 'pdo_sqlite', 'memory' => true]);

$store = new DoctrineDbalMessageStore('symfony', $connection);
$store->setup();

$chat = new Chat(new Agent($platform, 'mistral-small-latest'), $store);
$messages = new MessageBag(Message::forSystem('You are a helpful assistant. You only answer'));

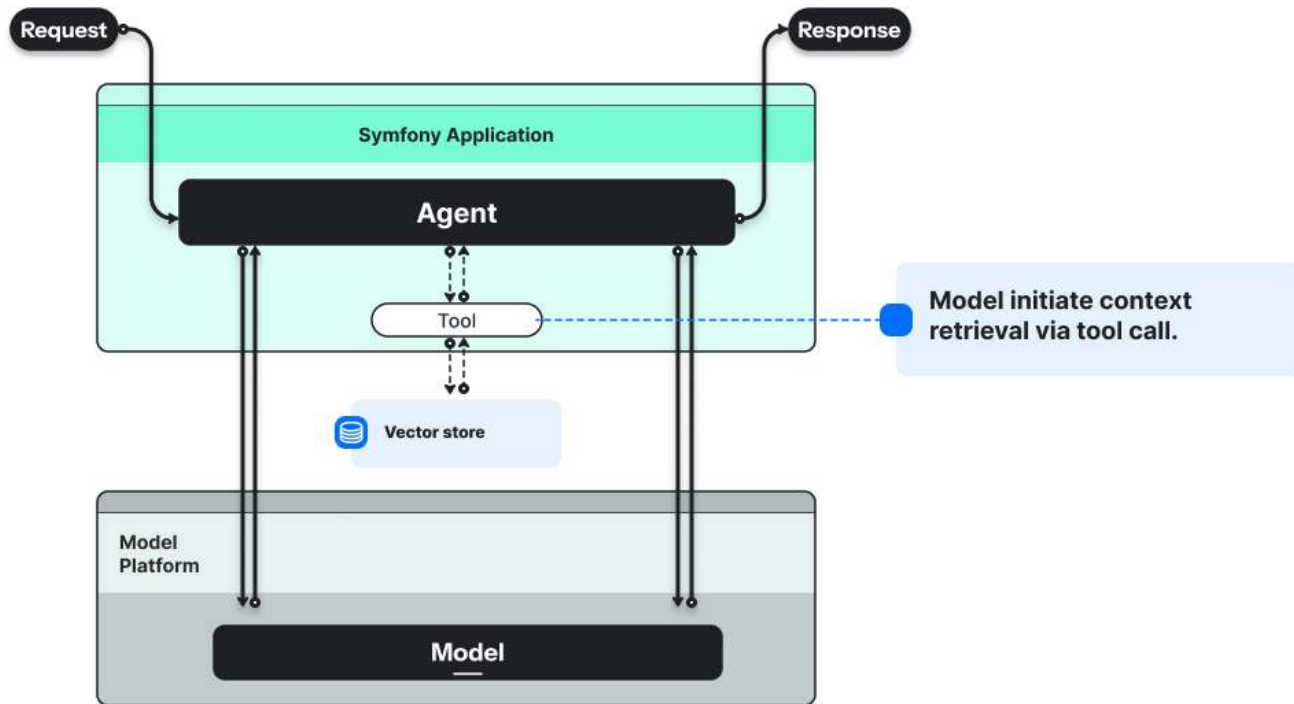
$chat->initiate($messages);
$chat->submit(Message::ofUser('My name is Mathieu.'));
$message = $chat->submit(Message::ofUser('What is my name?'));

echo $message->getContent().\PHP_EOL; // "Your name is Mathieu."
```

## **RAG - Store Component**

---

# Retrieval-Augmented Generation





# Store Component

---

# Store Component

---

## Vector Store Abstraction Layer

 Lifecycle Management

 Adding & Querying for Documents

# Store Component

---

## Vector Store Abstraction Layer

 Lifecycle Management

 Adding & Querying for Documents

## Document Indexing Pipeline

 Loading

 Filtering

 Transforming

 Vectorizing

# Store Component

---

## Vector Store Abstraction Layer

 Lifecycle Management

 Adding & Querying for Documents

## Document Indexing Pipeline

 Loading

 Filtering

 Transforming

 Vectorizing

```
composer require symfony/ai-store
```

# Store Component

---

## Achieve semantic search.

Vectorizing a user prompt and searching for similar documents in a vector database.

Then, LLM can infer based on those private documents.

 Load →  Filter →  Transform →  Vectorize →  Store

# Store Bridges

---

**18+ Store Bridges available**



Azure Search



ChromaDB



Clickhouse



CloudFlare Vectorize



Manticore



MariaDB



Meilisearch



Milvus



MongoDB



Neo4J



OpenSearch



Pinecone



PostgreSQL (pgvector)



Qdrant



Redis



Supabase



SurrealDB



Typesense

# **AI Bundle**

---

# AI Bundle Installation

---

```
composer require symfony/ai-bundle
```



# AI Bundle Installation

---

```
composer require symfony/ai-bundle
```

Or use the Symfony Flex alias:

```
composer require ai
```

# AI Bundle Installation

---

```
composer require symfony/ai-bundle
```

Or use the Symfony Flex alias:

```
composer require ai
```

✨ **Seamlessly integrates** Platform, Store, and Agent components into Symfony applications

# Bundle Configuration

```
# config/packages/ai.yaml
ai:
    platform:
        openai: # or anthropic, mistral, huggingface
            api_key: '%env(OPENAI_API_KEY)%'

    agent:
        default:
            model: 'gpt-5-mini'
            prompt: 'You are a pirate and you write funny.'
            # more options ...

    store:
        chroma_db:
            symfonyblog:
                collection: 'symfony_blog'

# multi_agent, indexer, vectorizer ...
```

# Bundle Configuration

```
# config/packages/ai.yaml
ai:
    platform:
        openai: # or anthropic, mistral, huggingface
            api_key: '%env(OPENAI_API_KEY)%'

    agent:
        default:
            model: 'gpt-5-mini'
            prompt: 'You are a pirate and you write funny.'
            # more options ...

    store:
        chroma_db:
            symfonyblog:
                collection: 'symfony_blog'

# multi_agent, indexer, vectorizer ...
```

# Bundle Configuration

```
# config/packages/ai.yaml
ai:
    platform:
        openai: # or anthropic, mistral, huggingface
            api_key: '%env(OPENAI_API_KEY)%'

    agent:
        default:
            model: 'gpt-5-mini'
            prompt: 'You are a pirate and you write funny.'
            # more options ...

    store:
        chroma_db:
            symfonyblog:
                collection: 'symfony_blog'

# multi_agent, indexer, vectorizer ...
```

# Bundle Configuration

```
# config/packages/ai.yaml
ai:
  platform:
    openai: # or anthropic, mistral, huggingface
      api_key: '%env(OPENAI_API_KEY)%'

  agent:
    default:
      model: 'gpt-5-mini'
      prompt: 'You are a pirate and you write funny.'
      # more options ...

  store:
    chroma_db:
      symfonyblog:
        collection: 'symfony_blog'

# multi_agent, indexer, vectorizer ...
```

# Bundle Configuration

```
# config/packages/ai.yaml
ai:
    platform:
        openai: # or anthropic, mistral, huggingface
            api_key: '%env(OPENAI_API_KEY)%'

    agent:
        default:
            model: 'gpt-5-mini'
            prompt: 'You are a pirate and you write funny.'
            # more options ...

    store:
        chroma_db:
            symfonyblog:
                collection: 'symfony_blog'

# multi_agent, indexer, vectorizer ...
```

# Subagent as Tool

---

```
# config/packages/ai.yaml
ai:
  # Platform Configuration ...
  agent:
    researcher:
      model: 'gpt-4o'
      prompt: 'You do heavy research.'
    smalltalker:
      model: 'gpt-4o-mini'
      tools:
        # Agent as tool 🍷
        - agent: 'researcher'
          name: 'research_topic'
          description: 'Does research on a given topic.'
```





# Profiler

Search profilesLatest

Request / Response

Performance

Logs1

Events

Routing

Cache

Twig

Twig Components

HTTP Client3

AI Symphony AI3

MCP0

Serializer

Configuration

Forms

Exception

Debug

Profiler settings

```
Result    Tool call:

1. similarity_search(searchTerm: "latest Symfony version") (ID:
fc_0276048e15ec372b00699f0a4f51e081998339d90e4a119e4a)

Metadata
• token_usage:
  Symfony\AI\Platform\TokenUsage\TokenUsage {#1055 ▾
    -promptTokens: 279
    -completionTokens: 19
    -thinkingTokens: 0
    -toolTokens: null
    -cachedTokens: 0
    -remainingTokens: 199504
    -remainingTokensMinute: null
    -remainingTokensMonth: null
    -totalTokens: 298
  }
}
```

Call 2

Model text-embedding-ada-002

Input "latest Symfony version"

Options

Result 1. Vector with 1536 dimensions

Call 3

Model gpt-4o-mini

Input

1. System: # Conversation Memory

This is the memory I have found for this conversation. The me  
so try to answer utilizing the memory as much as possible. Yo  
memory. If the memory is irrelevant, ignore it. Do not reply  
not  
reference it as this is just for your reference.

# Current Symfony Versions:

LTS: 7.4.5

Latest: 8.0.5

In development: 8.1.0-DEV





# Profiler

Platform calls (input, output)

Search profilesLatest

Request / Response

Performance

Logs1

Events

Routing

Cache

Twig

Twig Components

HTTP Client3

AI Symphony AI3

MCP0

Serializer

Configuration

Forms

Exception

Debug

Profiler settings

ResultTool call:

1. `similarity_search(searchTerm: "latest Symfony version")` (ID: `fc_0276048e15ec372b00699f0a4f51e081998339d90e4a119e4a`)

Metadata

- token\_usage:
  - Symfony\AI\Platform\TokenUsage\TokenUsage {#1055
    - promptTokens: 279
    - completionTokens: 19
    - thinkingTokens: 0
    - toolTokens: null
    - cachedTokens: 0
    - remainingTokens: 199504
    - remainingTokensMinute: null
    - remainingTokensMonth: null
    - totalTokens: 298

Call 2

Modeltext-embedding-ada-002

Input"latest Symfony version"

Options

Result1. Vector with 1536 dimensions

Call 3

Modelgpt-4o-mini

Input

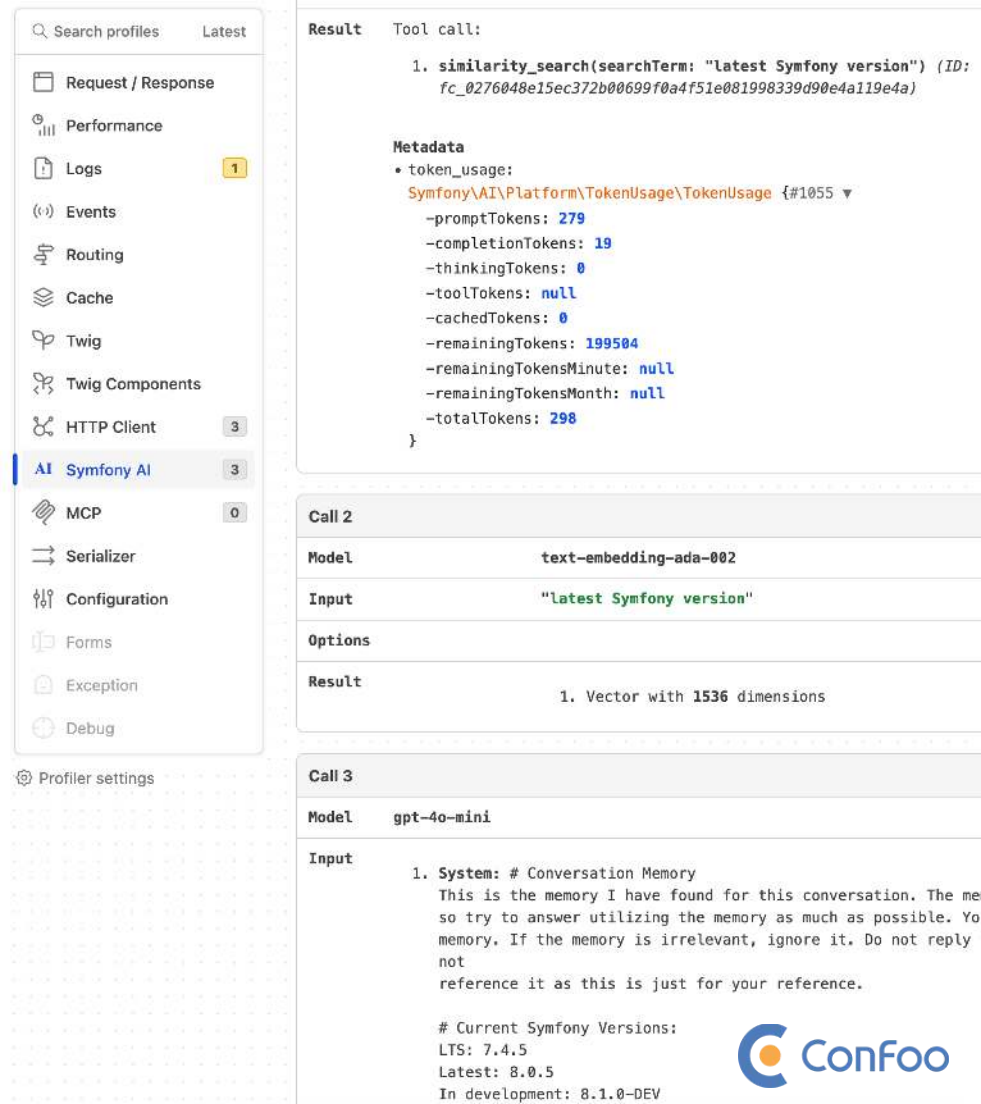
1. System: # Conversation Memory  
This is the memory I have found for this conversation. The me  
so try to answer utilizing the memory as much as possible. Yo  
memory. If the memory is irrelevant, ignore it. Do not reply  
not  
reference it as this is just for your reference.  
  
# Current Symfony Versions:  
LTS: 7.4.5  
Latest: 8.0.5  
In development: 8.1.0-DEV



# Profiler

Platform calls (input, output)

Tools calls (parameters, results)



The screenshot displays the Profiler application interface. On the left is a sidebar menu with various categories: Search profiles, Latest, Request / Response, Performance, Logs (with a yellow badge '1'), Events, Routing, Cache, Twig, Twig Components, HTTP Client (with a badge '3'), AI (selected, with a badge '3'), MCP (with a badge '0'), Serializer, Configuration, Forms, Exception, and Debug. Below the sidebar is a 'Profiler settings' section. The main content area shows a 'Tool call' result for a 'similarity\_search' function. The result is a JSON object containing an ID and a list of token usage metrics. Below this, there are two sections: 'Call 2' and 'Call 3'. 'Call 2' shows details for a 'text-embedding-ada-002' model, including its input and options. 'Call 3' shows details for a 'gpt-4o-mini' model, including its input and a list of current Symfony versions.

**Search profiles** Latest

**Request / Response**

**Performance**

**Logs** 1

**Events**

**Routing**

**Cache**

**Twig**

**Twig Components**

**HTTP Client** 3

**AI** **Symfony AI** 3

**MCP** 0

**Serializer**

**Configuration**

**Forms**

**Exception**

**Debug**

**Profiler settings**

**Tool call:**

```
1. similarity_search(searchTerm: "latest Symfony version") (ID: fc_0276048e15ec372b00699f0a4f51e081998339d90e4a119e4a)
```

**Metadata**

- token\_usage:
  - Symfony\AI\Platform\TokenUsage\TokenUsage {#1055
    - promptTokens: 279
    - completionTokens: 19
    - thinkingTokens: 0
    - toolTokens: null
    - cachedTokens: 0
    - remainingTokens: 199504
    - remainingTokensMinute: null
    - remainingTokensMonth: null
    - totalTokens: 298

**Call 2**

**Model** text-embedding-ada-002

**Input** "latest Symfony version"

**Options**

**Result** 1. Vector with 1536 dimensions

**Call 3**

**Model** gpt-4o-mini

**Input**

```
1. System: # Conversation Memory
This is the memory I have found for this conversation. The me
so try to answer utilizing the memory as much as possible. Yo
memory. If the memory is irrelevant, ignore it. Do not reply
not
reference it as this is just for your reference.

# Current Symfony Versions:
LTS: 7.4.5
Latest: 8.0.5
In development: 8.1.0-DEV
```

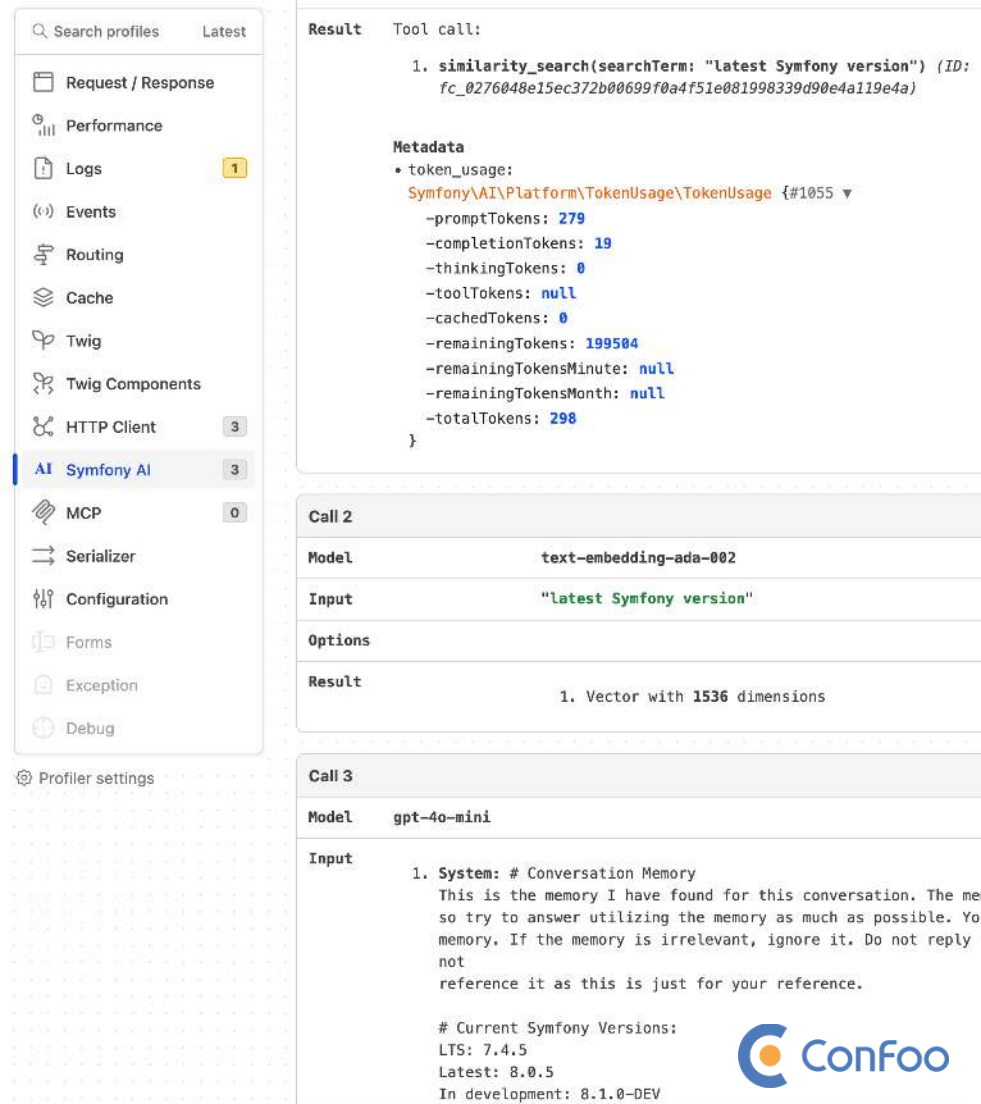


# Profiler

Platform calls (input, output)

Tools calls (parameters, results)

Metadata (consumed tokens, remaining tokens)



The screenshot displays the Profiler application interface. On the left is a sidebar menu with various categories: Request / Response, Performance, Logs (1), Events, Routing, Cache, Twig, Twig Components, HTTP Client (3), AI (Symfony AI, 3), MCP (0), Serializer, Configuration, Forms, Exception, and Debug. The 'AI' category is selected, showing 'Symfony AI' with a count of 3. The main content area shows a 'Tool call' result for a 'similarity\_search' function. The result is a JSON object containing metadata about token usage. Below this, there are two call detail sections: 'Call 2' and 'Call 3'. 'Call 2' shows a model 'text-embedding-ada-002' with an input 'latest Symfony version' and a result of a vector with 1536 dimensions. 'Call 3' shows a model 'gpt-4o-mini' with a complex input including system memory and current Symfony versions. The bottom right corner features the ConFoo logo.

```
Search profiles Latest
```

Request / Response

Performance

Logs 1

Events

Routing

Cache

Twig

Twig Components

HTTP Client 3

AI Symfony AI 3

MCP 0

Serializer

Configuration

Forms

Exception

Debug

```
Profiler settings
```

```
Result Tool call:

1. similarity_search(searchTerm: "latest Symfony version") (ID:
fc_0276048e15ec372b00699f0a4f51e081998339d90e4a119e4a)

Metadata
• token_usage:
  Symfony\AI\Platform\TokenUsage\TokenUsage {#1055 ▾
    -promptTokens: 279
    -completionTokens: 19
    -thinkingTokens: 0
    -toolTokens: null
    -cachedTokens: 0
    -remainingTokens: 199504
    -remainingTokensMinute: null
    -remainingTokensMonth: null
    -totalTokens: 298
  }
}
```

Call 2

|         |                                |
|---------|--------------------------------|
| Model   | text-embedding-ada-002         |
| Input   | "latest Symfony version"       |
| Options |                                |
| Result  | 1. Vector with 1536 dimensions |

Call 3

|       |                                                                                                                                                                                                                                                                                                                                                                                              |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model | gpt-4o-mini                                                                                                                                                                                                                                                                                                                                                                                  |
| Input | 1. System: # Conversation Memory<br>This is the memory I have found for this conversation. The me<br>so try to answer utilizing the memory as much as possible. Yo<br>memory. If the memory is irrelevant, ignore it. Do not reply<br>not<br>reference it as this is just for your reference.<br><br># Current Symfony Versions:<br>LTS: 7.4.5<br>Latest: 8.0.5<br>In development: 8.1.0-DEV |



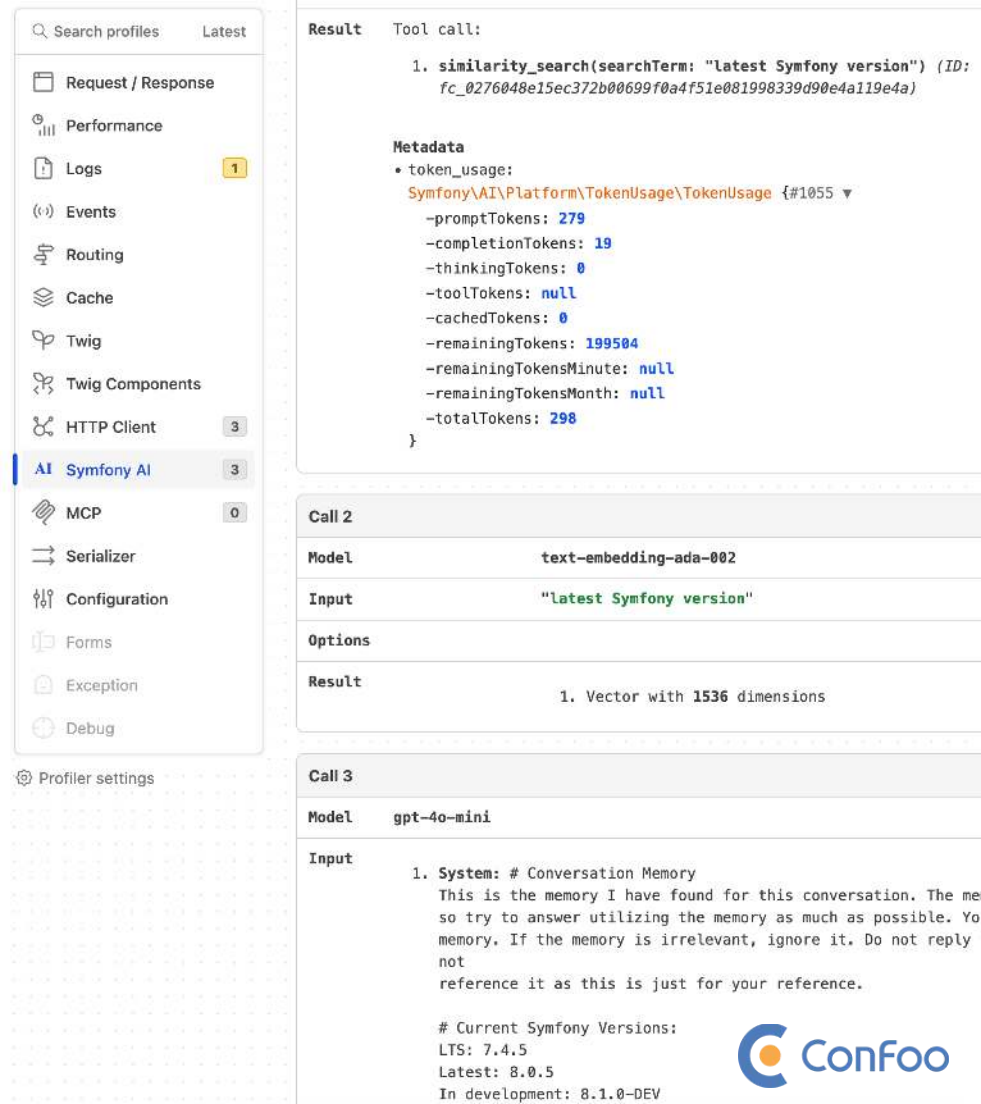
# Profiler

Platform calls (input, output)

Tools calls (parameters, results)

Metadata (consumed tokens, remaining tokens)

MCP Panel (tools, resources, prompts)



The screenshot displays the Profiler application interface. On the left is a sidebar menu with the following items: Search profiles (Latest), Request / Response, Performance, Logs (1), Events, Routing, Cache, Twig, Twig Components, HTTP Client (3), AI Symphony AI (3), MCP (0), Serializer, Configuration, Forms, Exception, and Debug. The 'AI Symphony AI' item is selected. The main panel shows the details for a tool call. It includes a 'Tool call:' section with a list of items, a 'Metadata' section with token usage details, and a 'Call 2' section with model, input, options, and result. The 'Call 3' section is also visible at the bottom.

**Tool call:**

1. `similarity_search(searchTerm: "latest Symfony version")` (ID: `fc_0276048e15ec372b00699f0a4f51e081998339d90e4a119e4a`)

**Metadata**

- token\_usage:
  - Symfony\AI\Platform\TokenUsage\TokenUsage {#1055 ▾
    - promptTokens: 279
    - completionTokens: 19
    - thinkingTokens: 0
    - toolTokens: null
    - cachedTokens: 0
    - remainingTokens: 199504
    - remainingTokensMinute: null
    - remainingTokensMonth: null
    - totalTokens: 298

**Call 2**

|         |                                |
|---------|--------------------------------|
| Model   | text-embedding-ada-002         |
| Input   | "latest Symfony version"       |
| Options |                                |
| Result  | 1. Vector with 1536 dimensions |

**Call 3**

|       |                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model | gpt-4o-mini                                                                                                                                                                                                                                                                                                                                                                                                    |
| Input | <p>1. <b>System:</b> # Conversation Memory</p> <p>This is the memory I have found for this conversation. The me so try to answer utilizing the memory as much as possible. Yo memory. If the memory is irrelevant, ignore it. Do not reply not reference it as this is just for your reference.</p> <p># Current Symfony Versions:</p> <p>LTS: 7.4.5</p> <p>Latest: 8.0.5</p> <p>In development: 8.1.0-DEV</p> |

# ✨ **Symfony AI Demo application**

---

 [github.com/symfony/ai-demo](https://github.com/symfony/ai-demo)



# Roadmap

---

# **Symfony AI Roadmap**

---



# Symfony AI Roadmap

---

**Stabilization**, code harmonization across components

# Symfony AI Roadmap

---

**Stabilization**, code harmonization across components

**1.0** release - [github.com/symfony/ai/issues/1301](https://github.com/symfony/ai/issues/1301)

# Symfony AI Roadmap

---

**Stabilization**, code harmonization across components

**1.0** release - [github.com/symfony/ai/issues/1301](https://github.com/symfony/ai/issues/1301)

Feature depth

# Symfony AI Roadmap

---

**Stabilization**, code harmonization across components

**1.0** release - [github.com/symfony/ai/issues/1301](https://github.com/symfony/ai/issues/1301)

Feature depth

Collaborations

# Symfony AI Roadmap

---

**Stabilization**, code harmonization across components

**1.0** release - [github.com/symfony/ai/issues/1301](https://github.com/symfony/ai/issues/1301)

Feature depth

Collaborations

Follow emerging **standards**

# What are your plans for Symfony AI?

---

Join the club of contributors & users!

Made with contrib.rocks.

# How to?

---

# How to?

---

1. Checkout ``symfony/ai`` repository



# How to?

---

1. Checkout ``symfony/ai`` repository
2. Explore ``examples/`` and ``demo/`` directories

# How to?

---

1. Checkout ``symfony/ai`` repository
2. Explore ``examples/`` and ``demo/`` directories
3. **Build your first feature** and give feedback

# How to?

---

1. Checkout ``symfony/ai`` repository
2. Explore ``examples/`` and ``demo/`` directories
3. **Build your first feature** and give feedback
4. ``AGENTS.md`` and ``CLAUDE.md`` files are provided to allow you working with AI agents!

# **Thank you!**

## **Questions?**



# SymfonyAI



Feedback